STATIC MEMBERS, STATIC CLASS, FRIEND FUNCTION, AND FRIEND CLASS

Conducted by

Mehedi Hasan

Lecturer, Dept. of CSE

1. STATIC CLASS MEMBERS

- Shared across all objects of a class
- Memory allocated only once for the class
- Accessed using ClassName::member
- Useful for common properties (e.g., interest rate, counters)

Real-life Example: Bank interest rate is same for all accounts.

STATIC MEMBER : COUNTING OBJECTS

```cpp
#include <iostream>
using namespace std;

class Student {
    static int count;
public:
    Student() { count++; }
    static void showCount() { cout << "Total Students: " << count << endl; }
};

int Student::count = 0;

int main() {
    Student s1, s2;     // Creates 2 students
    Student::showCount(); // Output: Total Students: 2

    Student s3;         // Creates another student
    Student::showCount(); // Output: Total Students: 3
}
```

STATIC MEMBER :BANK INTEREST RATE

```cpp
#include <iostream>
using namespace std;

class Bank {
    static float rate;
public:
    static void setRate(float r) { rate = r; }
    static void showRate() { cout << "Interest Rate: " << rate << "%" << endl; }
};

float Bank::rate = 5.0;  // default rate

int main() {
    Bank::showRate();     // Interest Rate: 5%
    Bank::setRate(7.5);   // change rate
    Bank::showRate();     // Interest Rate: 7.5%
    return 0;
}
```

EXAMPLE (DIFFICULT): UNIQUE EMPLOYEE ID

```cpp
#include <iostream>
using namespace std;

class Employee {
    static int nextID;
    int id;
public:
    Employee() { id = nextID++; }
    void showID() { cout << "Employee ID: " << id << endl; }
};

int Employee::nextID = 1000;
```

```cpp
int main() {
    Employee e1, e2, e3;
    e1.showID();  // Employee ID: 1000
    e2.showID();  // Employee ID: 1001
    e3.showID();  // Employee ID: 1002

    return 0;
}
```

2. STATIC CLASS (SIMULATED)

- C++ does not have static class like C#
- We simulate by making all methods static
- Useful for math libraries, helper functions

- Example: MathUtility::square(5)

3. FRIEND FUNCTION

- Not a member, but can access private data
- Declared inside class with keyword 'friend'
- Often used in operator overloading

- **Real-life Example: Allowing a special function to access ATM account balance.**

FRIEND FUNCTION :ACCESS PRIVATE MEMBER

```
#include <iostream>
using namespace std;
class base {
private:
   int private_variable;
protected:
   int protected_variable;
public:
   base() {
      private_variable = 10;
      protected_variable = 99;
   }
   // Friend function declaration
   friend void friendFunction(base& obj);
};
```

```
// friend function definition
void friendFunction(base& obj) {
   cout << "Private Variable: "
      << obj.private_variable << endl;
   cout << "Protected Variable: "
      << obj.protected_variable;
}
```

```
int main() {
   base object1;
   friendFunction(object1);
   return 0;
}
```

FRIEND FUNCTION :ACCESS PRIVATE MEMBER WITH STATIC MEMBER

```
class base {
private:
    static int static_var;
public:
    friend void friendFunction();
};
int base::static_var = 42;

void friendFunction() {
    cout << "Static Variable: " << base::static_var;
}
int main() {
    friendFunction(); // works without passing an object
    return 0;
}
```

N.B:
If a class has a static member, it belongs to the class itself, not to an object.

FRIEND FUNCTION

- **Non-static members** → Need an object to access, even in a friend function.
- **Static members** → Can be accessed directly by the friend function without an object.

4. FRIEND CLASS

- A class can grant another class access to its private data
- Declared using: friend class ClassName;
- Useful in strong relationships (e.g., Car ↔ Engine)

- **Real-life Example: A Bank Manager has full access to a Customer Account.**

EXAMPLE: CAR AND ENGINE

```
#include <iostream>
using namespace std;

// Engine class
class Engine {
private:
   int horsepower;

public:
   Engine(int hp) : horsepower(hp) {}

   // Declare Car as a friend class
   friend class Car;
};
```

```
// Car class
class Car {
private:
   Engine engine;

public:
   Car(int hp) : engine(hp) {}

   void showHorsepower() {
      // Accessing private member
of Engine
      cout << "Car engine
horsepower: " <<
engine.horsepower << endl;
   }
};
```

```
int main() {
   Car myCar(250);
myCar.showHorsepower();

   return 0;
```

SUMMARY

- Static Member: Shared property (Bank interest)
- Static Function: Operates only on static data
- Static Class: Utility library (Math functions)
- Friend Function: Special function access (ATM)
- Friend Class: Strong relationship (Bank Manager ↔ Account)